Homework 1

Part A

- It is 12. The length of the chromosome = the # of tasks.
- It's $8^{12} = 68,719,476,736$.
- Because binary encoding is usually used to solve binary tasks and tasks with continuous values. If we use binary encoding here, the single point crossover may "cut up" the 3-bit IDs, causing meaningless ID or abnormal mutations. Plus, using 3-bit binary encoding will lengthen the chromosome representation from 12 to 36, causing computation overhead.
- As these tasks are conflicting. If we optimize one objective, the others may worsen. For example, if we only consider to maximize the skill match, we may overload some of the experts while others remain idle.

Part B

Check the attached `.py` file

Part C

Table of Population Size

Population Size	Fitness	Gen. Convergence
30	0.6039	81
50	0.6104	49
80 (baseline)	0.6120	49
100	0.6116	85
200	0.6136	54

Table of Mutation Rate

Mutation Rate	Fitness	Gen. Convergence
0.01	0.6025	56
0.05	0.6104	48
`1 / NUM_TASKS`	0.6039	81
0.1	0.6061	61
0.3	0.6045	82

Observations

Based on the data listed above, I think population size has a bigger impact on the **quality** (as we are talking about the quality, I consider *fitness* as a more important factor) of the result. We can see the range of fitness:

$$Range(pop) = 0.6136 - 0.6039 = 0.0097$$

$$Range(m\_rate) = 0.6104 - 0.6025 = 0.0079$$

Reason: A small population means a limited gene pool. The algorithm quickly makes all individuals to look identical, leading to “inbreeding” and getting stuck in local optima. With a larger population, more diverse genetic combinations are possible, and there will be higher chance to find out a better combination as the solution. Though mutation rate also has impact on the quality, its upper and lower limits are still constrained by the size of the population. Excessively high mutation rates can disrupt the optimal gene combinations (Building Blocks) that have already been discovered. This is precisely why you say it is “**constrained by population size**”—when the population itself lacks sufficient diversity, merely increasing the mutation rate will cause the algorithm to degenerate into pure random walk behavior, preventing stable convergence to high-quality solutions.

Part D

1. We can locally (or, dynamically) fix the problem. Instead of repeating the process from scratch, we can keep the good part and only fix the broken part. To be more specific, we need to update the data first, i.e., remove the guy from `DEVELOPERS`. Then, for the current state, if there is a task being allocated to the removed person, randomly re-allocate the task to the rest of the developers, and continue the process. In this way, we can keep the "good genes" while maintaining the efficiency.
2. We can firstly add a penalty to the function such that if there is a pair of tasks with dependencies which are allocated to different persons, the penalty will increase. Plus, we need NSGA-II, as it is hard to use the current strategy, weight-sum method, to find a proper set of weights due to the increased number of objectives, which introduces more severe conflicts. Also, it can provide a SET of solutions (Pareto-optima), allowing the user (manager) to see the trade-offs and select the most proper solution themselves.